Database II Lab/ 3rd Grade

[Second Semester] and [2026]

[Lab 6] [24/3/ 2026] [8.30 + 10.30 am]

Instructor Information

Instructor
Dr. Rasool Hisham
Dr. Zainab Namh
Dr. Azhar Flaih
Assist. Prof. Zahraa Jaaz

Email
[rasool.hisham@nahrainuniv.edu.iq]

Hours
[2 Hrs]

DATA BINDING WITH UI CONTROLS

C# · MySQL · ADO.NET · MySqlDataReader

Lab Objectives

By the end of this lab, students will be able to:

- Create a MySQL database schema with two related tables: contact_category and contacts.
- Build a C# Windows Forms application connected to MySQL using ADO.NET.
- Use **MySqlDataReader** to read and display contact records in a ListBox.
- Populate a ComboBox a ListBox from the database using **Anonymous object binding** method.
- Implement CRUD: Read (display in ListBox), Save (insert), and Delete.

Part 1 — Database Setup

1.1 Create the Schema

Open MySQL Workbench and execute:

```
CREATE DATABASE IF NOT EXISTS phone_book;
USE phone_book;
```

1.2 Table: contact_category

contact_category

ContactCategoryID INT | PK | NOT NULL | AUTO_INCREMENT
CategoryName VARCHAR(45) | NOT NULL

```
// Seed sample categories
INSERT INTO contact_category (CategoryName)
VALUES ('Family'), ('Friend'), ('Work'), ('Other');
```

1.3 Table: contacts

Contacts

ContactId INT | PK | NOT NULL | AUTO_INCREMENT
ContactName VARCHAR(45) | NOT NULL
ContactTel VARCHAR (25)| NOT NULL
ContactImg BLOB | NULL
ContactCategID INT | NOT NULL → FK to contact_category.ContactCategoryID

```
// Insert sample data to test with
INSERT INTO contacts (ContactName, ContactTel, ContactCategID)
VALUES ('Alice',7701234567,1),
       ('Bob',7809876543,3),
       ('Carol',7755553333,2);
```

Part 2 — Visual Studio Project Setup

2.1 Create the Project

1. Open Visual Studio.
2. File → New → Project → Windows Forms App
3. Name the project: PhoneBookApp → Create.

2.2 Install MySQL Connector via NuGet

4. Go to: Tools → NuGet Package Manager → Manage NuGet Packages for Solution.
5. Search for MySql.Data and install it.
6. At the top of your form file, add:

```
     using MySql.Data.MySqlClient;
```

Part 3 — Windows Forms UI Design

3.1 Controls to Add

Add the following controls to Form1. Set the Name property exactly as shown:

Control	Name Property	Text / Label	Purpose
Label	—	Contact Name:	label
TextBox	TxtName		Required — contact name
Label	—	Phone:	label
TextBox	TxtTel		Required — phone number

Control	Name Property	Text / Label	Purpose
Label	—	Category:	label
ComboBox	CboCateg		Required — bound to contact_category
Button	BtnRead	Read	Load contacts → ListBox
Button	BtnAdd	Save	Insert new contact
Button	BtnDelete	Delete	Delete the currently displayed contact
ListBox	LstContacts		Displays all loaded contacts

Part 4 — C# Code

> ⚠ **Important Note**
> All code below uses exactly the same pattern as the previous lab manuals: connectionString declared and assigned in the class, or assigned in form_Load, and all database operations wrapped in try { } catch (Exception ex) { }.

4.1 Declare and assign the Connection String in the Class

Declare **connectionString** as a class-level field — **not** inside any method:

```
namespace PhoneBookApp
{
    public partial class Form1 : Form
    {
        string connectionString="";   // declared and assigned here

        public Form1()
        {
            InitializeComponent();
        }
    }
}
```

4.2 Form_Load —Populate ComboBox

The **anonymous binding method** used is a technique where you create an object "in the runtime" (without a formal class like Contact or Employee) to store two different pieces of information in a single ComboBox or ListBox item:

1. **The Display Text:** What the user sees (e.g., a Category Name).
2. **The Hidden Value:** What the database needs (e.g., CategoryID).

How CboCategory binding work

```
CboCateg.DisplayMember = "Text";   → this is the custom property name "Text.
This property holds the string shown to the user (CategoryName)

CboCateg.ValueMember   = "Value";   → this is another custom property name
"Value". This property is the hidden ID (ContactCategoryID)from the database

CboCateg.Items.Add(new { Text = ..., Value = ... });
```

→ anonymous object

4.3 BtnRead_Click — Read Contacts and Display in ListBox

This follows the same reader pattern as with the combo box. **MySqlDataReader** reads each row; each row is added to LstContacts.

Step 1: Prepare the ListBox UI

Before adding any new data, you must reset the control so it knows how to handle the "On the fly" objects.

- Clear all existing items from the ListBox to prevent duplicate entries. **LstContacts.Items.Clear();**
- Set the **DisplayMember** property to **"Text"**.
- Set the **ValueMember** property to **"Value"**.

Step 2: Establish the Database Connection, define the SQL query and execute the Data Reader

Step 3: Process the Rows (The While Loop)

Iterate through the results retrieved by the Reader.

- Use a **While loop** with the condition reader.Read().
- Inside the loop, extract the values from the columns (ID, Name, Tel, Category) and store them in local string or int variables.

Step 4: Create and Bind the Anonymous Object

This is the "On the Fly" binding method from the manual.

- Add a **new anonymous object** to the ListBox items.
- Define a **Text** property: Combine the ID, Name, and Category into one long string for display.
- Define a **Value** property: Assign the raw integer **ID** from the database. This allows the Delete button to find the record later.

4.4 BtnAdd_Click — Insert a New Contact

This button reads what the student typed into the text boxes and saves it as a new row in the contacts table.

Step 1: Open the database connection using connString

Use the class-level variable connString that was assigned in Form_Load. Wrap the connection in a using block so it closes automatically.

Step 2: Write the INSERT query with @parameter placeholders

Define the SQL INSERT statement as a string. Use @name, @phone, and @Category as placeholders — never put the actual values directly into the string. See Lab 5.

Step 3: Bind the text box values to the parameters with AddWithValue

Replace each @placeholder with the actual text the user typed. Use AddWithValue for each parameter.

```
cmd.Parameters.AddWithValue("@name",TxtName.Text.Trim());
cmd.Parameters.AddWithValue("@phone",TxtTel.Text.Trim());
```

However, for the ComboBox:

How to Retrieve the Hidden ID

This is the most important part. Since the object has no official "Type" name, you cannot cast it as a String or Int. You must use the **dynamic** keyword, which tells C# to figure out the properties at runtime. Casts the selected item so we can access its .Value and .Text properties.

```
// 1. Cast the selected item to 'dynamic'
var selectedItem = (dynamic)CboCateg.SelectedItem;

// 2. Access the 'Value' property we created earlier
int id = selectedItem.Value;

// 3. Use that ID in your INSERT or UPDATE query
cmd.Parameters.AddWithValue("@id", id);
```

Step 4: Execute the query and check rowsAffected

Call `ExecuteNonQuery()` to run the INSERT. It returns the number of rows added. Check `rowsAffected > 0` to confirm success. See Lab 5.

4.7 BtnDelete_Click — Delete the Currently Displayed Contact

This button deletes whichever contact the student has selected in the ListBox.

Step 1: Check that the student actually selected something in the ListBox

```
// Check if item is selected
if (LstContacts.SelectedItem == null)
{
    MessageBox.Show("Please select a contact to delete");
    return;
}
```

Step 2: Read the ContactId from the selected ListBox item using dynamic

Each item in LstContacts was added as an **anonymous object** — exactly the same pattern used to populate CboCateg in the lab. Because the item is an anonymous object, we use **`dynamic`** to access its properties at runtime.

```
// Retrieve selected ID using dynamic
var selectedItem = (dynamic)LstContacts.SelectedItem;
int id = selectedItem.Value;
```

Step 3: Open the database connection, Write the DELETE query, bind the parameter, and execute the Delete command

Write a parameterized DELETE query using **`@id`** as a placeholder. Then bind the actual value using **`AddWithValue`** — See Lab 5.

Step 4: Execute the DELETE, show confirmation, then reload the ListBox

Call **`ExecuteNonQuery()`** to run the DELETE statement, show a success message.